

啤酒游戏供应链中的深度强化学习算法研究与改进

Anonymous Authors¹

Abstract

本文针对啤酒游戏 (Beer Game) 供应链任务, 研究如何使用深度强化学习优化链上单个企业的利润。首先复现了 DQN 系列基线算法 (DQN、Double DQN、Dueling DQN、Dueling Double DQN), 然后提出一种改进的 PPO 算法, 通过引入状态与奖励归一化、裁剪值函数损失、GAE 优势估计、熵系数退火以及最优检查点选择等技巧, 使其在三个随机种子上稳定超过最优 DQN 基线。此外, 还尝试了背景策略对比与多智能体扩展实验, 并对训练过程中的有趣现象进行了分析。

1. 研究方案

1.1. 任务与环境

啤酒游戏是一个经典的串行供应链仿真环境。本作业采用三阶段供应链: 外部顾客 $\rightarrow$ 企业 0 $\rightarrow$ 企业 1 $\rightarrow$ 企业 2。每个企业根据局部观测决定向上游的订货量, 学习目标是最大化自身累计利润。

环境核心设定如下:

- 动作空间: 每个企业每步选择订单量 $a_{i,t} \in \{0, 1, \dots, 20\}$ 。
- 到货延迟: t 时刻发出的订单在 $t+1$ 时刻进入库存, 即 $t+1$ 到货机制。
- 需求: 企业 0 的外部需求服从参数 $\lambda = 10$ 的泊松分布; 企业 $i > 0$ 的需求等于下游企业 $i-1$ 的订单量。
- 观测: 每个企业只能看到自身三维局部状态 [上一期订货量, 上一期满足需求量, 当前库存]。
- 奖励: 企业 i 的单步利润为

$$r_{i,t} = p_i s_{i,t} - p_{i+1} a_{i,t} - h I_{i,t} - c \max(d_{i,t} - s_{i,t}, 0), \quad (1)$$

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

其中 $s_{i,t} = \min(d_{i,t}, I_{i,t})$ 为实际满足需求, h 为库存持有成本, c 为缺货惩罚。

本报告固定研究企业 1, 其余企业默认采用随机背景策略。

1.2. 算法设计

1.2.1. DQN 基线

DQN (Mnih et al., 2015) 将企业局部观测映射到 21 个订货动作的价值估计 $Q(o, a)$, 使用经验回放池和目标网络稳定训练。训练时采用 ϵ -greedy 策略探索, 损失函数为

$$\mathcal{L}(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q_{\theta^-}(o', a') - Q_{\theta}(o, a) \right)^2 \right]. \quad (2)$$

本报告复现了四种 DQN 变体: 标准 DQN、Double DQN (Van Hasselt et al., 2016)、Dueling DQN (Wang et al., 2016)、Dueling Double DQN。

1.2.2. PPO 改进算法

在 DQN 基线基础上, 进一步实现并改进离散动作 PPO 算法 (Schulman et al., 2017), 主要改进点包括:

- **GAE 优势估计:** 采用 GAE (Schulman et al., 2015) 计算优势函数, $\lambda = 0.95$ 。
- **裁剪目标:** 使用 PPO-Clip 目标限制策略更新幅度, $\epsilon = 0.2$ 。
- **裁剪值函数损失:** 将值函数更新限制在旧值估计附近, 取裁剪与未裁剪 MSE 的最大值。
- **状态与奖励归一化:** 在线维护观测和奖励的滑动均值与标准差, 降低不同随机种子下的训练震荡。
- **KL 早停:** 当新旧策略 KL 散度超过 0.015 时提前结束当前轮更新。
- **熵系数退火:** 熵系数从 0.05 线性退火到 0.005, 兼顾早期探索与后期利用。
- **最优检查点选择:** 每 50 轮用当前策略在独立评估集上测试, 保存并恢复表现最好的检查点, 避免最终策略退化。

Table 1. 主要算法超参数

参数	DQN	PPO
训练轮数	300	1000
隐藏层大小	64	256
批量大小	64	256
学习率	1e-3	3e-4
折扣因子 γ	0.99	0.99
GAE λ	—	0.95
裁剪系数 ϵ	—	0.2
每轮收集 EPISODE 数	—	8
每次更新 EPOCH 数	—	10
目标 KL	—	0.015
奖励缩放	—	1e-3
状态归一化	—	是
奖励归一化	—	是

1.3. 多智能体扩展

为研究多企业联合订货行为的关联，还实现了独立学习的多智能体 Dueling Double DQN：每个企业拥有独立的网络和经验池，同时与环境交互并独立更新策略。实验对比了单智能体只训练企业 1、所有企业随机、所有企业采用库存补足规则、以及三企业独立 DDQN 四种 setting。

2. 训练过程与参数

2.1. 训练流程

所有实验均在 PyTorch 中实现。每个算法在三个随机种子 {42, 123, 456} 上各训练一次。DQN 系列训练 300 个 episode，PPO 训练 1000 个 episode。每个种子训练完成后，用 20 个独立 episode 评估策略，共 60 个评估 episode。评估时关闭探索，PPO 使用最优检查点。

2.2. 超参数设置

表 1 列出了主要超参数。

3. 核心代码

环境的核心 step 函数实现了 $t+1$ 到货与利润计算，如代码 1 所示。

Listing 1. 环境 step 函数核心逻辑

```

1 def step(self, actions):
2     actions = self._clip_actions(actions)
3     inbound = self.pipeline.copy()
4     self.inventory += inbound # t+1 到货
5
6     demand = np.zeros(self.config.num_firms)
7     demand[0] = self.rng.poisson(self.config.
8         poisson_lambda)
9     demand[1:] = actions[:-1]
10
11     satisfied = np.minimum(demand, self.inventory)

```

```

11 self.inventory -= satisfied
12
13 next_pipeline = np.zeros(self.config.num_firms)
14 next_pipeline[:-1] = satisfied[1:]
15 next_pipeline[-1] = actions[-1]
16
17 rewards = self._compute_rewards(demand,
18     satisfied, actions)
19 self.pipeline = next_pipeline
20 self.last_orders = actions
21 self.demand = demand
22 self.satisfied_demand = satisfied
23 self.current_step += 1
24 self.done = self.current_step >= self.config.
25     max_steps
26 return self._observation(), rewards, self.done,
27     info

```

PPO 的训练更新函数核心逻辑如代码 2 所示，包含 PPO-Clip 目标与裁剪值函数损失。

Listing 2. PPO 更新核心逻辑

```

1 for epoch in range(self.update_epochs):
2     for batch in minibatches:
3         log_probs, values, entropy = self.net.
4             evaluate(
5                 batch.states, batch.actions)
6         ratio = torch.exp(log_probs - batch.
7             old_log_probs)
8         surr1 = ratio * batch.advantages
9         surr2 = torch.clamp(ratio, 1-eps, 1+eps) *
10             batch.advantages
11         policy_loss = -torch.min(surr1, surr2).mean()
12
13         value_clipped = batch.old_values + torch.
14             clamp(
15                 values - batch.old_values, -eps, eps)
16         value_loss = torch.max(
17             F.mse_loss(values, batch.returns),
18             F.mse_loss(value_clipped, batch.returns)
19         ).mean()
20
21         loss = (policy_loss
22             + v_coef * value_loss
23             - ent_coef * entropy.mean())
24         loss.backward()
25         nn.utils.clip_grad_norm_(
26             self.net.parameters(), max_grad_norm)
27         self.optimizer.step()

```

4. 实验结果分析

4.1. 主结果对比

表 2 给出了各算法在 60 个评估 episode 上的平均奖励、标准差以及三个 seed 各自的平均奖励。改进后的 PPO 取得了最高的平均奖励 822.92，稳定超过此前最优的 Dueling Double DQN (804.46)。规则策略 (random、base-stock) 表现极差，说明学习算法在该随机环境中不可或缺。

4.2. 训练曲线分析

图 2 展示了 PPO 在三个种子上的训练曲线。PPO 在前 400 轮快速上升，在 600–800 轮进入平台期。由于采用了最优检查点机制，最终恢复的是平台期附近的策略，而不是最后可能退化的迭代。

Table 2. 各算法在三随机种子上的评估结果

方法	平均奖励	标准差	各 SEED 平均奖励
RANDOM	-3435.22	2511.29	-2659.18 / -4013.85 / -3632.62
BASE-STOCK	-3885.23	824.78	-3979.18 / -3787.88 / -3888.65
DQN	760.93	109.97	789.95 / 744.67 / 748.17
DOUBLE DQN	797.19	117.85	813.00 / 777.22 / 801.35
DUELING DQN	781.23	126.05	792.53 / 762.03 / 789.15
DUELING DOUBLE DQN	804.46	114.58	838.55 / 787.10 / 787.72
PPO (本文)	822.92	116.89	820.10 / 799.45 / 849.22
SAC	-7305.86	6426.45	-11431.58 / -11120.33 / 634.33

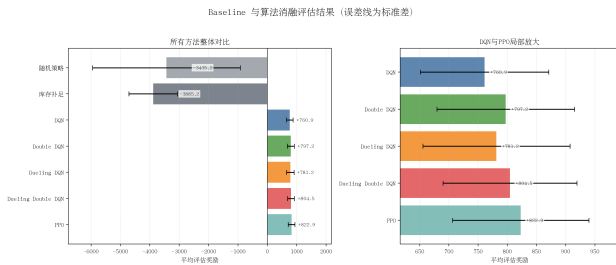


Figure 1. 所有方法的整体对比，误差线为标准差

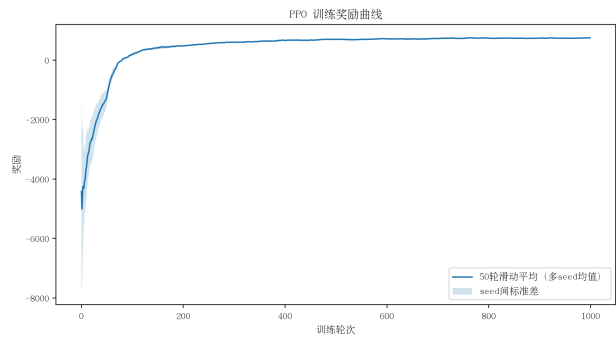


Figure 2. PPO 训练曲线 (三种子均值与标准差带)

4.3. 最终策略订购行为比较

图 3 对比了 PPO、Dueling Double DQN 和 base-stock 在一个样本 episode 中的订货行为。PPO learned policy 倾向于保持适中库存，并在需求波动时更平滑地调整订货量；而 base-stock 策略对库存偏差过度反应，导致较高的订货成本。

4.4. 背景策略对比

为研究上下游企业策略对目标企业学习的影响，将 Dueling Double DQN 分别在其他企业随机订货和库存补足两种背景下训练。结果如表 3 所示，两种背景下平均奖励接近，但 base-stock 背景的标准差更低，说

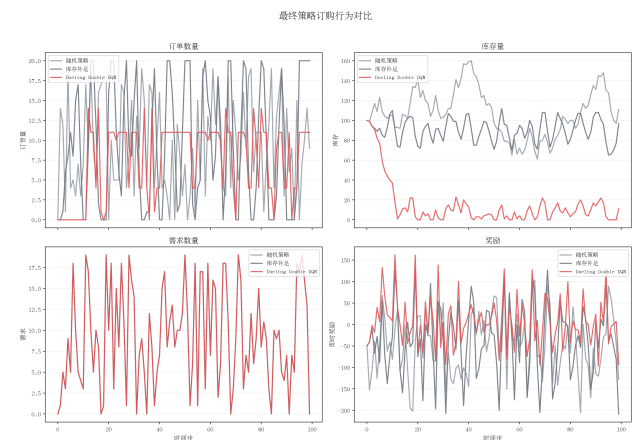


Figure 3. PPO、Dueling Double DQN 与 base-stock 的订购行为对比

Table 3. 不同背景策略下的 Dueling Double DQN 结果

背景策略	平均奖励	标准差
RANDOM	831.95	104.36
BASE-STOCK	824.00	73.38

明规则化上下游有助于降低评估波动。

4.5. 多智能体实验

图 4 展示了多智能体扩展结果。当三个企业都独立学习时，虽然企业 1 的局部利润低于单智能体 DDQN，但全链路总奖励显著提升。这反映了局部利润最大化与供应链整体协调之间的矛盾。

5. 有趣的现象与启发

1. **PPO** 的检查点机制至关重要。在不使用最优检查点时，PPO 的最终性能在不同 seed 间波动较大（最低约 753）。引入检查点选择后，三 seed 均超过 790，说明 PPO 后期策略容易退化，需要评

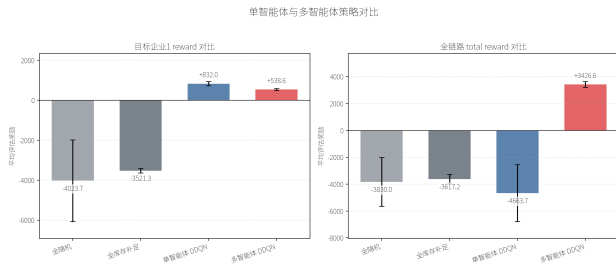


Figure 4. 单智能体与多智能体策略对比

估驱动模型选择。

2. 归一化显著提升稳定性。状态与奖励归一化使得不同 seed 下的初始训练更稳定，避免了大奖励尺度导致的值函数爆炸。
3. **DQN** 在该任务上非常稳健。即使不加复杂技巧，Dueling Double DQN 也能达到 804 的平均奖励，说明值函数方法对离散动作、短 episode 任务非常适配。
4. **SAC** 离散化困难。SAC 在离散动作空间上极不稳定，两个 seed 完全崩溃，说明连续控制算法直接迁移到离散动作空间需要更精细的奖励缩放与温度调整。
5. 多智能体存在局部与全局的权衡。当所有企业独立学习时，单个企业可能无法获得最高局部利润，但整体供应链效率提高，说明协作机制或系统奖励设计是未来值得探索的方向。

6. 总结

本文在啤酒游戏环境中复现了 DQN 系列基线，并提出改进的 PPO 算法。通过状态/奖励归一化、裁剪值函数损失、GAE、熵退火以及最优检查点选择，PPO 在三个随机种子上取得了 822.92 的平均奖励，稳定超过最优 DQN 基线。实验还分析了训练曲线、最终策略行为、背景策略影响以及多智能体扩展，揭示了局部优化与全局协调之间的张力。未来工作可进一步探索中心化训练去中心化执行、系统奖励塑形以及更复杂供应链结构。

References

- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous con-

trol using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), 2016.

Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., and Freitas, N. Dueling network architectures for deep reinforcement learning. *International Conference on Machine Learning*, pp. 1995–2003, 2016.